

Problem 1

Given a Binary Search Tree and a range [low, high]. Find all the numbers in the BST that lie in the given range.

Note: Element greater than or equal to root go to the right side.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

// Create new node
struct Node* createNode(int val) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->data = val;
    node->left = node->right = NULL;
    return node;
}

// Insert into BST
struct Node* insert(struct Node* root, int val) {
    if (root == NULL)
        return createNode(val);

    if (val < root->data)
        root->left = insert(root->left, val);
    else
        root->right = insert(root->right, val);
    return root;
}

// Print nodes in given range
```

```

void printInRange(struct Node* root, int low, int high) {
    if (root == NULL)
        return;
    if (root->data > low)
        printInRange(root->left, low, high);
    if (root->data >= low && root->data <= high)
        printf("%d ", root->data);
    if (root->data < high)
        printInRange(root->right, low, high);
}

int main() {
    struct Node* root = NULL;
    // Creating BST
    root = insert(root, 17);
    root = insert(root, 4);
    root = insert(root, 18);
    root = insert(root, 2);
    root = insert(root, 9);
    int low = 4, high = 24;
    // Your Details
    printf("Name: Divya Choudhary\n");
    printf("PRN: 25070521069\n");
    printf("Output: ");
    printInRange(root, low, high);
    return 0;
}

```

Output

Name: Divya Choudhary

PRN: 25070521069

Output: 4 9 17 18

=== Code Execution Successful ===

Problem 2

Given a string *s*, composed of different combinations of '(', ')', '{', '}', '[', ']'. Determine whether the Expression is balanced or not. An expression is balanced if:

1. Each opening bracket has a corresponding closing bracket of the same type.
2. Opening brackets must be closed in the correct order.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX 1000
struct Stack {
    char arr[MAX];
    int top;
};
void init(struct Stack *s) {
    s->top = -1;
}
void push(struct Stack *s, char c) {
    if (s->top == MAX - 1) return;
    s->arr[++(s->top)] = c;
}
```

```

char pop(struct Stack *s) {
    if (s->top == -1) return '\0';
    return s->arr[(s->top)--];
}

int isMatching(char open, char close) {
    if (open == '(' && close == ')') return 1;
    if (open == '{' && close == '}') return 1;
    if (open == '[' && close == ']') return 1;
    return 0;
}

int isBalanced(char *expr) {
    struct Stack s;
    init(&s);

    for (int i = 0; expr[i]; i++) {
        char ch = expr[i];
        if (ch == '(' || ch == '{' || ch == '[') {
            push(&s, ch);
        }
        else if (ch == ')' || ch == '}' || ch == ']') {
            char top = pop(&s);
            if (!isMatching(top, ch)) {
                return 0;
            }
        }
    }

    return (s.top == -1);
}

int main() {
    char expr[MAX];
    printf("Enter expression: ");
    scanf("%s", expr);

```

```
if (isBalanced(expr))
    printf("Output: true\n");
else
    printf("Output: false\n");
printf("\nName: Divya Chooudhary\n");
printf("PRN: 25070521069\n");
return 0;
}
```

•

Output
Enter expression: ([[]]){}
Output: true
Name: Divya Chooudhary
PRN: 25070521069
=== Code Execution Successful ===